

Introduction to Data Science and Engineering

- Text mining



Some materials courtesy of
Rafael A. Irizarry, and are modified
from the original version.

Zhenqin (Michael) Wu / 吳楨欽

School of Computing and Data Science
University of Hong Kong

Slide deck originally created by RB Luo



Todd Vaziri ✓

@tvaziri

Every non-hyperbolic tweet is from iPhone (his staff).

Every hyperbolic tweet is from Android (from him).

Data scientist David Robinson conducted an analysis to determine if data supported this assertion, let's go through his analysis and learn some basic text mining skills

<http://varianceexplained.org/r/trump-tweets/>



Donald J. Trump @realDonaldTrump

Good luck #TeamUSA

#OpeningCeremony #Rio2016

pic.twitter.com/mS8qsQpJPh

27,391 Likes

8,392 Retwee

Aug 5, 2016 at 8:59 PM

via **Twitter for iPhone**



Donald J. Trump @realDonaldTrump

Heading to New Hampshire - will be talking about Hillary saying her brain SHORT CIRCUITED, and other thing!

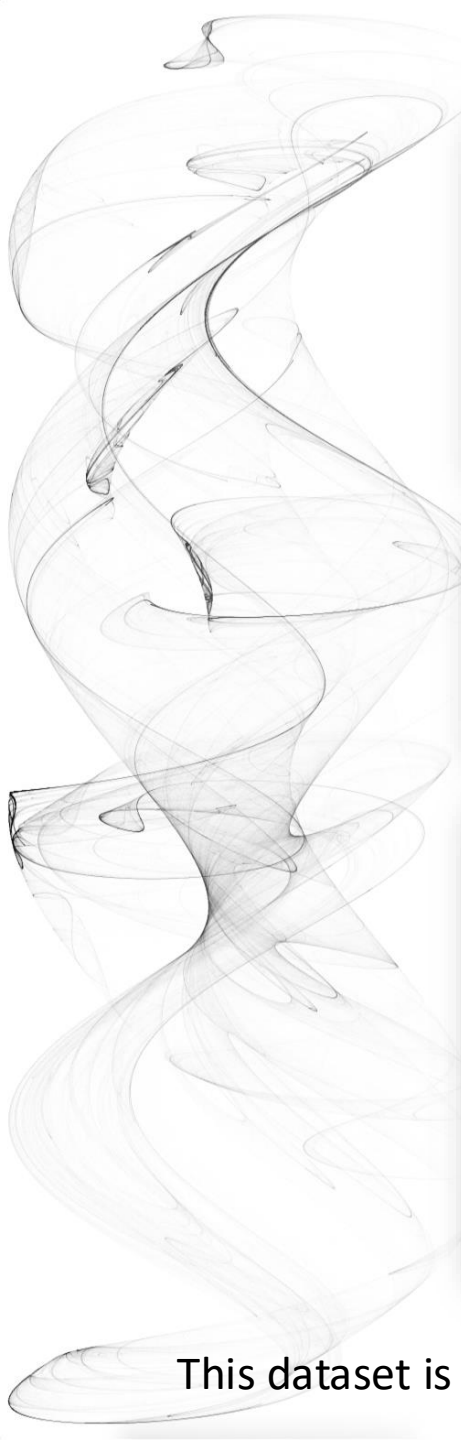
4,451 Likes

1,480 Retwee

Aug 6, 2016 at 11:11 AM

via **Twitter for Android**

<https://twitter.com/tvaziri/status/762005541388378112>



```
library(tidyverse)
library(lubridate)
library(tidytext)
library(textdata)

all_tweets <- jsonlite::fromJSON("Lec12-trump_tweets.json",
                                simplifyDataFrame = TRUE)
```

```
> all_tweets |> select(-text) |> head(all_tweets, n=3)
```

	id	isRetweet	isDeleted	device	favorites	retweets	date	isFlagged
1	9.845497e+16	f	f	TweetDeck	49	255	2011-08-02 18:07:48	f
2	1.234653e+18	f	f	Twitter for iPhone	73748	17404	2020-03-03 01:34:50	f
3	1.218011e+18	t	f	Twitter for iPhone	0	7396	2020-01-17 03:22:47	f

```
> all_tweets |> pull(text) |> head(3) |> str_wrap(width = 80) |> cat(sep="\n===\n")
```

Republicans and Democrats have both created our economic problems.

===

I was thrilled to be back in the Great city of Charlotte, North Carolina with thousands of hardworking American Patriots who love our Country, cherish our values, respect our laws, and always put AMERICA FIRST! Thank you for a wonderful evening!! #KAG2020 <https://t.co/dNJZfRs19y>

===

RT @CBS_Herridge: READ: Letter to surveillance court obtained by CBS News questions where there will be further disciplinary action and cho...

This dataset is downloaded from: <https://www.thetrumparchive.com/faq>

```
# Filter out retweets and tweets start with quote symbols
trump_tweets <- all_tweets |>
  filter(isRetweet == "f" & !str_detect(text, '^[\"\\']')) |>
  mutate(date = ymd_hms(date))

# Count devices
trump_tweets |> count(device) |> arrange(desc(n)) |> head(5)
```

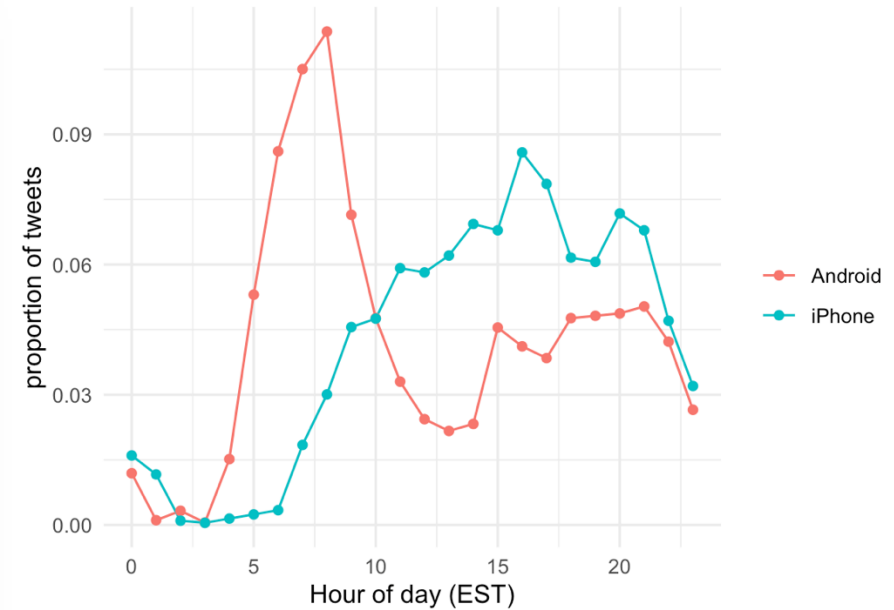
```
# Select tweets from iPhone/Android during the campaign period
campaign_tweets <- trump_tweets |>
  extract(device, "source", "Twitter for (.*)") |>
  filter(source %in% c("Android", "iPhone") &
    date >= ymd("2015-06-17") &
    date < ymd("2016-11-08")) |>
  arrange(date)
```

	device	n
1	Twitter for iPhone	18026
2	Twitter Web Client	10700
3	Twitter for Android	4652
4	TweetDeck	466
5	Twitter Media Studio	353

- We are selecting tweets what were posted between the day Trump announced his campaign and the election day.
- Use `ymd_hms` (from `lubridate`) to parse dates.
- Use `extract` to identify device where tweets were posted.

When are these tweets posted?

```
# Plot and examine time in the day
# when tweets are posted from the two platforms
campaign_tweets |>
  mutate(hour = hour(with_tz(date, "EST"))) |>
  group_by(source) |>
  count(hour) |>
  mutate(percent = n / sum(n)) |>
  ungroup() |>
  ggplot(aes(hour, percent, color = source)) +
    geom_line() +
    geom_point() +
    scale_y_continuous() + #labels = percent_format() +
    labs(x = "Hour of day (EST)", y = "proportion of tweets", color = "") +
    theme_minimal()
```



- Will you tweet/X a bunch at 8AM?
- Which line fits better a normal person's daily schedule?

Text as data

```
> poem <- c("Roses are red,", "Violets are blue,",  
+           "Sugar is sweet,", "And so are you.")  
> example <- data_frame(line = c(1, 2, 3, 4),  
+                        text = poem)  
> example  
# A tibble: 4 × 2  
  line text  
  <dbl> <chr>  
1     1 Roses are red,  
2     2 Violets are blue,  
3     3 Sugar is sweet,  
4     4 And so are you.
```

```
> example |> unnest_tokens(word, text)  
# A tibble: 13 × 2  
  line word  
  <dbl> <chr>  
1     1 roses  
2     1 are  
3     1 red  
4     2 violets  
5     2 are  
6     2 blue  
7     3 sugar  
8     3 is  
9     3 sweet  
10    4 and  
11    4 so  
12    4 are  
13    4 you
```

Here we used a function `unnest_tokens` (from `tidytext`) to extract all words from the input text corpora.

This function creates a new column (“word”) containing tokens from the source column (“text”), also expanding other columns, if any.

Tokenization: splitting text into units

- Sentence-level
- Word-level
- Subword-level: byte-pair encoding, WordPiece

This is the entry to Natural Language Processing (NLP)? Which type of tokenization is used by LLMs?



Text as data

```
# An example tweet
example_tweet <- campaign_tweets[3008,]
example_tweet |> pull(text) |> str_wrap(width = 65) |> cat()

# Use regex to remove links and tokenize the example tweet
links_re <- "https://t.co/[A-Za-z\\d]+!&"
example_tweet |>
  mutate(text = str_replace_all(text, links_re, "")) |>
  unnest_tokens(word, text) |>
  pull(word)
```

```
Great to be back in Iowa! #TBT with @JerryJrFalwell joining me in
Davenport- this past winter. #MAGA https://t.co/A5IF0QHnic
```

[1]	"great"	"to"	"be"	"back"	"in"	"iowa"
[7]	"tbt"	"with"	"jerryjrfalwell"	"joining"	"me"	"in"
[13]	"davenport"	"this"	"past"	"winter"	"maga"	

We used regular expression here to remove all links in the tweets



Text as data

```
all_campaign_tweet_words <- campaign_tweets |>
  mutate(text = str_replace_all(text, links_re, "")) |>
  unnest_tokens(word, text)

# The most common words
all_campaign_tweet_words |>
  count(word) |>
  top_n(10, n) |>
  arrange(desc(n))
```

	word	n
1	the	2323
2	to	1398
3	and	1238
4	in	1177
5	i	1149
6	a	1117
7	you	995
8	of	975
9	is	941
10	on	874

- What are the most commonly used words?
- Does that surprise you?

The top words are often meaningless, referred to as stop words.

We can use a database of these commonly used words to exclude them from subsequent analysis: `stop_words` from `tidytext`.



Text as data

```
# Remove occurrences of stop words
all_campaign_tweet_words <- campaign_tweets |>
  mutate(text = str_replace_all(text, links_re, "")) |>
  unnest_tokens(word, text) |>
  filter(!word %in% stop_words$word)

# The most common words
all_campaign_tweet_words |>
  count(word) |>
  top_n(10, n) |>
  arrange(desc(n))
```

	word	n
1	hillary	400
2	trump2016	400
3	amp	324
4	people	303
5	makeamericagreatagain	293
6	america	254
7	clinton	224
8	poll	211
9	crooked	205
10	trump	184

- “trump” (184) + “trump2016” (400) > “hillary” (400)

```
# Remove pure numbers
all_campaign_tweet_words_no_number <- all_campaign_tweet_words |>
  filter(!str_detect(word, "^[\\d.]+$"))
```



Text as data

```
# Count the occurrences of each word by platform
all_campaign_tweet_words_no_number |>
  group_by(source) |>
  count(word) |>
  pivot_wider(names_from = "source", values_from = "n", values_fill = 0) |>
  top_n(5, Android + iPhone)
```

Here we are counting the occurrences of each word by the two platforms we want to compare.

`pivot_wider` is used to combine the two rows for the same word (but different platforms) back to one row.

```
# A tibble: 10 × 3
```

	word <chr>	Android <int>	iPhone <int>
1	america	114	140
2	amp	64	260
3	clinton	136	88
4	crooked	156	49
5	hillary	290	110
6	people	195	108
7	poll	117	94
8	trump	112	72
9	trump2016	3	397
10	makeamericagreatagain	0	293

Odds ratio

```
# Calculate the occurrences of each word and its odd ratio
word_by_source_odd_ratio <- all_campaign_tweet_words_no_number |>
  group_by(source) |>
  count(word) |>
  pivot_wider(names_from = "source", values_from = "n", values_fill = 0) |>
  mutate(odd_ratio = ((Android + 0.5) / (sum(Android) - Android + 0.5)) /
                ((iPhone + 0.5) / (sum(iPhone) - iPhone + 0.5)))

# Top words by platforms
top_android_words <- word_by_source_odd_ratio |>
  filter(Android + iPhone > 100) |>
  arrange(desc(odd_ratio)) |>
  head(10)

top_iphone_words <- word_by_source_odd_ratio |>
  filter(Android + iPhone > 100) |>
  arrange(odd_ratio) |>
  head(10)
```

Odds ratio (OR) is a summary statistic useful for quantifying the strength of the association between two events:

- Event 1 (E1): use of a specific word
- Event 2 (E2): use Android

Odds ratio =

Odds of E1 given E2 /

Odds of E1 given not E2

The (+0.5) part will be explained later in the course.

Odds ratio

```
# Calculate the occurrences of each word and its odd ratio
word_by_source_odd_ratio <- all_campaign_tweet_words_no_number |>
  group_by(source) |>
  count(word) |>
  pivot_wider(names_from = "source", values_from = "n", values_fill = 0) |>
  mutate(odd_ratio = ((Android + 0.5) / (sum(Android) - Android + 0.5)) /
    ((iPhone + 0.5) / (sum(iPhone) - iPhone + 0.5)))
```

```
# Top words by platforms
```

```
top_android > top_android_words
```

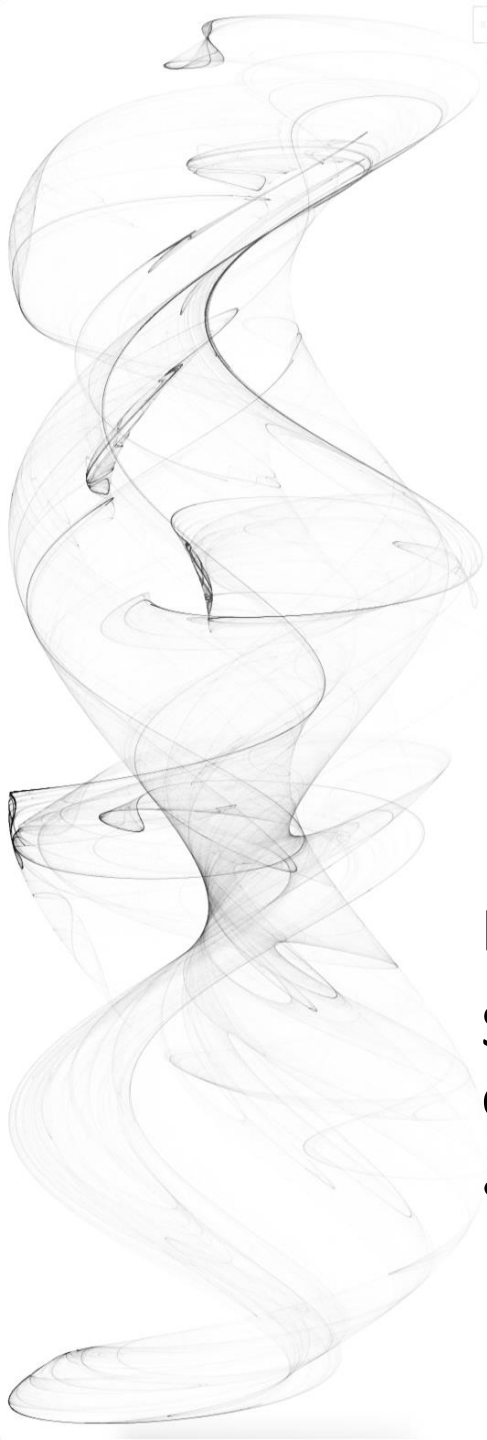
```
filter(Android > 100) # A tibble: 10 x 4
```

	word	Android	iPhone	odd_ratio
	<chr>	<int>	<int>	<dbl>
1	bad	104	26	3.36
2	crooked	156	49	2.70
3	cnn	116	37	2.65
4	ted	86	28	2.58
5	interviewed	76	25	2.55
6	media	78	26	2.52
7	hillary	290	110	2.25
8	president	84	33	2.14
9	cruz	115	46	2.11
10	win	74	30	2.08

```
> top_iphone_words
```

```
# A tibble: 10 x 4
```

	word	Android	iPhone	odd_ratio
	<chr>	<int>	<int>	<dbl>
1	makeamericagreatagain	0	293	0.00141
2	trump2016	3	397	0.00725
3	join	1	157	0.00798
4	amp	64	260	0.207
5	tomorrow	24	99	0.208
6	vote	46	66	0.592
7	america	114	140	0.689
8	tonight	70	83	0.715
9	iowa	62	65	0.808
10	poll	117	94	1.05



Insights

Since the frequently used words are different, it is very likely that the tweets are from different users.



How can we dig deeper?

Sentiment analysis: an NLP task focusing on analyzing the emotional tone expressed in a text corpus.

- Sentiments: positive, negative, trust, anger, joy, disgust, etc.



Sentiment analysis

Here we use a simple word-to-sentiment mapping available from `tidytext`
`get_sentiments` provides different mappings

- “bing”: positive / negative
- “afinn”: a score between -5 and 5
- “nrc”: different emotions

```
# Sentiment analysis  
nrc <- get_sentiments("nrc") |>  
  select(word, sentiment)  
  
nrc |> slice_sample(n = 10)
```

```
# A tibble: 10 × 2  
  word      sentiment  
  <chr>    <chr>  
1 friendly trust  
2 failure sadness  
3 antipathy negative  
4 terrorism negative  
5 disgruntled anger  
6 aftertaste negative  
7 probity positive  
8 distracting anticipation  
9 bacterium disgust  
10 duplicity negative
```

Sentiment analysis

```
# Add sentiment annotation to each word
word_by_source_odd_ratio <- word_by_source_odd_ratio |>
  inner_join(nrc, by = "word", relationship = "many-to-many")

word_by_source_odd_ratio |>
  arrange(desc(Android + iPhone)) |>
  head(10)
```

```
# A tibble: 10 × 5
```

	word <chr>	Android <int>	iPhone <int>	odd_ratio <dbl>	sentiment <chr>
1	poll	117	94	1.05	trust
2	trump	112	72	1.32	surprise
3	join	1	157	0.00798	positive
4	time	90	51	1.49	anticipation
5	bad	104	26	3.36	anger
6	bad	104	26	3.36	disgust
7	bad	104	26	3.36	fear
8	bad	104	26	3.36	negative
9	bad	104	26	3.36	sadness
10	tomorrow	24	99	0.208	anticipation

Sentiments (emotion) of the top words.

Note that we used a “many-to-many” relationship in `inner_join`

Sentiment analysis

```
# Calculate the occurrences of sentiments (according to words)
sentiment_by_source <- word_by_source_odd_ratio |>
  group_by(sentiment) |>
  summarize(Android_s = sum(Android), iPhone_s = sum(iPhone)) |>
  mutate(Android_s_r = Android_s / (sum(Android_s) - Android_s) ,
         iPhone_s_r = iPhone_s / (sum(iPhone_s) - iPhone_s)) |>
  mutate(odd_ratio = Android_s_r / iPhone_s_r) |>
  arrange(odd_ratio)
```

```
# A tibble: 10 × 6
```

	sentiment <chr>	Android_s <int>	iPhone_s <int>	Android_s_r <dbl>	iPhone_s_r <dbl>	odd_ratio <dbl>
1	positive	1827	1465	0.219	0.280	0.782
2	trust	1248	969	0.140	0.169	0.827
3	joy	695	534	0.0733	0.0866	0.846
4	anticipation	917	687	0.0990	0.114	0.867
5	surprise	530	340	0.0550	0.0535	1.03
6	fear	799	466	0.0852	0.0748	1.14
7	sadness	901	504	0.0972	0.0814	1.19
8	negative	1657	909	0.195	0.157	1.24
9	anger	962	516	0.104	0.0835	1.25
10	disgust	639	308	0.0670	0.0482	1.39

Group all words by their sentiments and sum their counts

Sentiments and their odds ratio between the two platforms:

Higher means more occurrences in Android.

Sentiment analysis

```
# Odd Ratio, log(OR) and its SE
```

```
sentiment_by_source <- sentiment_by_source |>  
  mutate(log_or = log(odd_ratio),  
         se = sqrt(1/Android_s + 1/(sum(Android_s) - Android_s) +  
                  1/iPhone_s + 1/(sum(iPhone_s) - iPhone_s)),  
         conf.low = log_or - qnorm(0.975)*se,  
         conf.high = log_or + qnorm(0.975)*se)
```

```
# Plot the distribution of sentiment ORs
```

```
sentiment_by_source |>  
  mutate(sentiment = reorder(sentiment, log_or)) |>  
  ggplot(aes(x = sentiment, ymin = conf.low, ymax = conf.high)) +  
    geom_errorbar() +  
    geom_point(aes(sentiment, log_or)) +  
    ylab("Log odds ratio for association between Android and sentiment") +  
    coord_flip()
```

One more (optional) step: add estimates of uncertainty. This aspect will be covered in future lectures.

Sentiment analysis

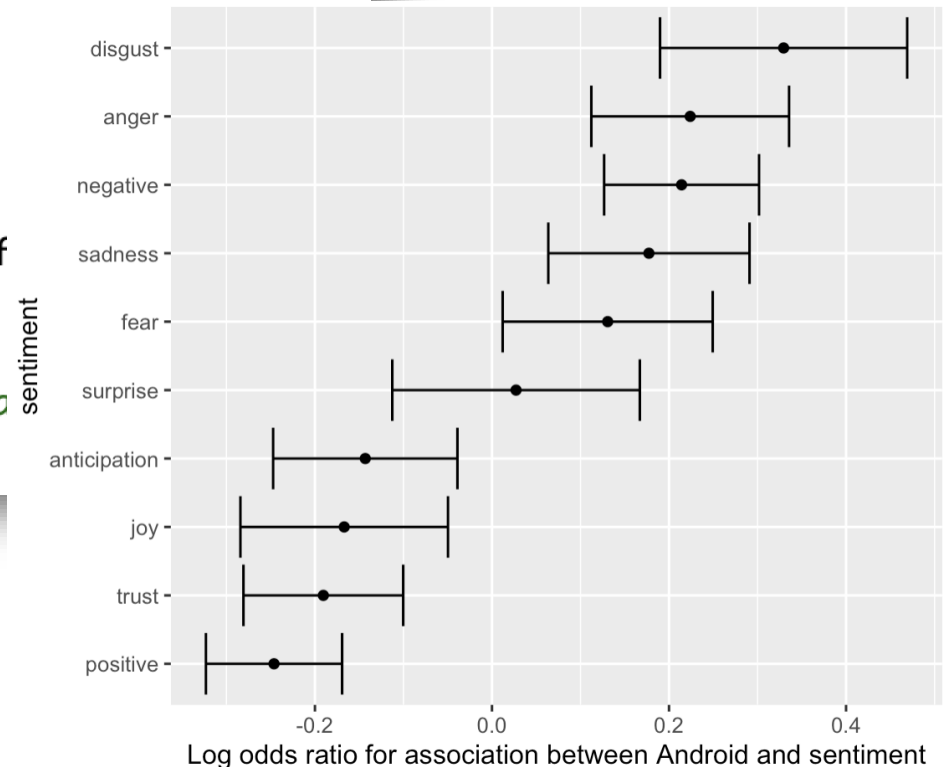
```
# Odd Ratio, log(OR) and its SE
```

```
sentiment_by_source <- sentiment_by_source |>
  mutate(log_or = log(odd_ratio),
         se = sqrt(1/Android_s + 1/(sum(Android_s) - Android_s) +
                  1/iPhone_s + 1/(sum(iPhone_s) - iPhone_s)),
         conf.low = log_or - qnorm(0.975)*se,
         conf.high = log_or + qnorm(0.975)*se)
```

```
# Plot the distribution of sentiment ORs
```

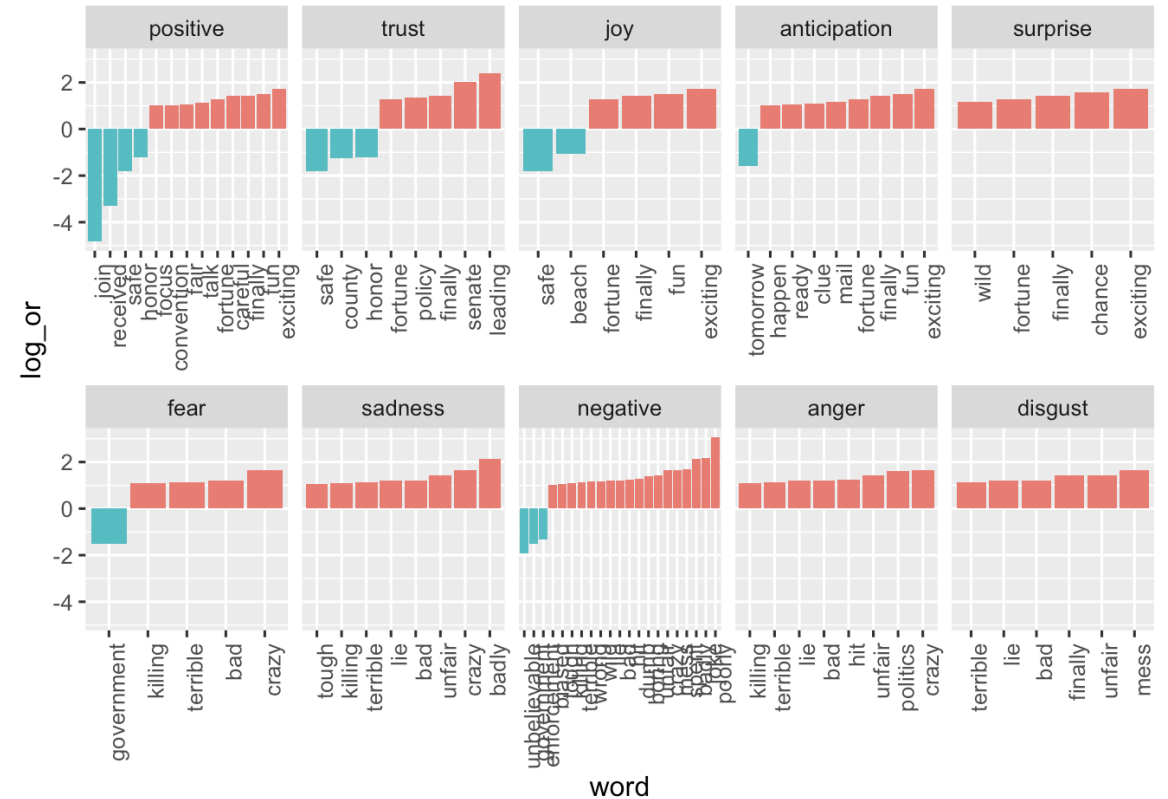
```
sentiment_by_source |>
  mutate(sentiment = reorder(sentiment, log_or)) |>
  ggplot(aes(x = sentiment, ymin = conf.low, ymax = conf
  geom_errorbar() +
  geom_point(aes(sentiment, log_or)) +
  ylab("Log odds ratio for association between Android c
  coord_flip()
```

We see that the negative emotions are associated with Android usage (for tweets).



Sentiment analysis

```
# Plot the top words for each sentiment
word_by_source_odd_ratio |>
  mutate(sentiment = factor(
    sentiment, levels = sentiment_by_source$sentiment)) |>
  mutate(log_or = log(odd_ratio)) |>
  filter(Android + iPhone > 10 & abs(log_or)>1) |>
  mutate(word = reorder(word, log_or)) |>
  ggplot(aes(word, log_or, fill = log_or < 0)) +
  facet_wrap(~sentiment, scales = "free_x", nrow = 2) +
  geom_bar(stat="identity", show.legend = FALSE) +
  theme(axis.text.x = element_text(angle = 90, hjust = 1))
```



Specific words that drive these differences.



Summary

- We examined a large collection of tweets through:
 - Looking at relevant metadata:
 - When were they posted?
 - Looking at the tweet contents:
 - Tokenize all tweets and remove irrelevant words
 - Calculate the occurrences and odds ratio of words in different devices
 - Sentiment analysis of the contents using word-to-sentiment mapping
- To learn more:
 - `tidytext` tutorial:
 - <https://www.tidytextmining.com/tidytext>
 - Some more twitter datasets to play around with:
 - <https://www.kaggle.com/datasets/crowdflower/twitter-airline-sentiment>
 - <https://www.kaggle.com/datasets/kazanova/sentiment140>
 - <https://www.kaggle.com/datasets/gpreda/covid19-tweets>